GUIDE DE TP : Concentrateur de Données de Vol

Systèmes Temps Réel & Avionique (Linux RT, C++ SCHED_FIFO, Python)

🎯 Objectifs Pédagogiques

- Mettre en œuvre un système d'exploitation Linux temps réel grâce au patch `PREEMPT_RT`.
- Développer une application critique en C++ utilisant des threads POSIX avec une politique d'ordonnancement déterministe (`SCHED_FIFO`).
- Implémenter un mécanisme de communication inter-processus par Mémoire Partagée (Shared Memory).
- Concevoir un superviseur en Python pour exploiter les données à haute fréquence.

🛠️ Matériel et Environnement (Toolchain)

- **Partie Matérielle :** Aucune (100% PC / Machine Virtuelle).
- **OS & Virtualisation :** VirtualBox, image ISO Ubuntu avec noyau patché `PREEMPT_RT`.
- **Environnement de développement :** VS Code (extensions C/C++ et Python), compilateurs `gcc/g++`, et Make/CMake.

📝 Étape par Étape : Travaux Pratiques

TP 1 : Validation de l'environnement Linux RT

Objectif : S'assurer que le noyau temps réel est opérationnel et comprendre l'environnement de travail.

1. Démarrer votre machine virtuelle ou poste sous le noyau `PREEMPT_RT`.
2. Vérifier la version du noyau dans le terminal via la commande : `uname -r` (doit mentionner `rt`).
3. Ouvrir VS Code et créer un dossier de travail pour le projet.

TP 2 : Le Producteur Haute Fréquence (C++ & Threads POSIX)

Objectif : Simuler un calculateur de vol générant des données de manière critique et déterministe.

1. Créer un fichier `producer.cpp`.
2. Écrire un thread POSIX configuré avec la politique `SCHED_FIFO` pour garantir la priorité temps réel.
3. Simuler l'acquisition de données avioniques (altitude, vitesse, cap) à haute fréquence (ex: toutes les 10 millisecondes).

TP 3 : Allocation de la Mémoire Partagée (IPC)

Objectif : Établir le canal de communication inter-processus ultra-rapide.

1. Dans votre code C++, utiliser les fonctions POSIX de gestion de la mémoire partagée (`shm_open`, `ftruncate`, `mmap`).
2. Définir une structure de données fixe (`FlightDataPacket`) contenant vos variables de vol.
3. À chaque cycle d'horloge, le producteur écrit les nouvelles valeurs directement dans cette zone mémoire.

TP 4 : Le Superviseur (Python)

Objectif : Lire les données de vol en temps réel depuis le processus Python sans perturber le C++.

1. Créer un script `supervisor.py`.
2. Utiliser les modules Python adaptés (`mmap` et `struct`) pour ouvrir et lire le segment de mémoire partagée créé par le C++.
3. Afficher en continu les flux de données reçus dans la console ou simuler un tableau de bord textuel.

TP 5 : Intégration et Validation

Objectif : Compiler l'ensemble, automatiser le build et tester le comportement global.

1. Écrire un fichier `CMakeLists.txt` pour compiler proprement le programme C++.
2. Lancer simultanément le programme C++ (producteur temps réel) et le script Python (superviseur).
3. Vérifier la fluidité des flux de données et l'absence de dérive temporelle.

> 💡 **Note de fin de TP :** Une fois ces 5 étapes validées, vous disposerez d'un concentrateur de données de vol fonctionnel et prêt à intégrer votre portfolio d'ingénierie temps réel pour le secteur aéronautique.